

HDOC 25

NAME = ARAYANA KHARE

BATCH = 15

1. Algorithm

Main Program

1. Start.
2. Declare integer `choice`.
3. Display the menu:
 - Decreasing right-angled triangle
 - Decreasing natural-number pattern
 - Decreasing repeated-number pattern
 - Reverse-number triangle
 - Exit
4. Read `choice`.
5. Use `switch(choice)`.
6. Call the corresponding parameterless function.
7. If the choice is invalid, display "Invalid choice!".
8. Repeat the menu until the user selects Exit.
9. Stop.

Pattern 1 — Decreasing Right-Angled Triangle

1. Input `n`.
2. Check whether `n > 0`.
3. Use an outer loop from `n` down to 1.
4. For each row, use an inner loop to print `*` the required number of times.
5. Move to the next line.

For `n = 4`, the required pattern is:

```
* * * *
* * *
* *
*
```

Pattern 2 — Decreasing Natural-Number Pattern

1. Input `n`.
2. Check whether `n > 0`.
3. Run the outer loop from `n` down to 1.
4. For every row, print natural numbers from 1 up to the current row value.
5. Move to the next line.

For `n = 4`:

```

1 2 3 4
1 2 3
1 2
1

```

Pattern 3 — Decreasing Repeated-Number Pattern

1. Input n.
2. Check whether $n > 0$.
3. Run the outer loop from n down to 1.
4. Print the current row value exactly that many times.
5. Move to the next line.

For n = 4:

```

4 4 4 4
3 3 3
2 2
1

```

Pattern 4 — Reverse-Number Triangle

1. Input n.
2. Check whether $n > 0$.
3. Run the outer loop from n down to 1.
4. In each row, print numbers from the current row value down to 1.
5. Move to the next line.

For n = 4:

```

4 3 2 1
3 2 1
2 1
1

```

2. Test-Case Table

Test

Choice

Input

n Expected Result Type

1 1 4 Decreasing * triangle with 4
rows

Valid

2 2 4 1 2 3 4 decreasing to 1 Valid

3 3 4 4 4 4 4 decreasing to 1 Valid

4 4 4 4 3 2 1 decreasing to 1 Valid
5 1 1 *
Boundar
y
6 2 0 Error: rows must be positive Invalid
7 3 -2 Error: rows must be positive Invalid
8 7 — Invalid choice! Invalid
9 5 — Program exits Valid

3. Complete C Program

```
#include <stdio.h>
void decreasingTriangle(void);
void naturalPattern(void);
void repeatedPattern(void);
void reverseTriangle(void);
int main(void)
{
    int choice;
    do
    {
        printf("\n===== DECREASING PATTERNS =====\n");
        printf("1. Decreasing right-angled triangle\n");
        printf("2. Decreasing natural-number pattern\n");
        printf("3. Decreasing repeated-number pattern\n");
        printf("4. Reverse-number triangle\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1:
                decreasingTriangle();
                break;
            case 2:
                naturalPattern();
                break;
```

```

case 3:
repeatedPattern();
break;
case 4:
reverseTriangle();
break;
case 5:
break;
printf("Exiting program...\n");
default:
printf("Invalid choice! Please enter 1 to
5.\n");
}
} while (choice != 5);
return 0;
}
/* Pattern 1 */
void decreasingTriangle(void)
{
int n, i, j;
printf("Enter number of rows: ");
scanf("%d", &n);
if (n <= 0)
{
printf("Error: Number of rows must be positive.\n");
return;
}
{
for (i = n; i >= 1; i--)
for (j = 1; j <= i; j++)
{
printf("* ");
}
printf("\n");
}
}

```

```

}
/* Pattern 2 */
void naturalPattern(void)
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);
    if (n <= 0)
    {
        printf("Error: Number of rows must be positive.\n");
        return;
    }
    {
        for (i = n; i >= 1; i--)
        for (j = 1; j <= i; j++)
        {
            printf("%d ", j);
        }
        printf("\n");
    }
}
/* Pattern 3 */
void repeatedPattern(void)
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);
    if (n <= 0)
    {
        printf("Error: Number of rows must be positive.\n");
        return;
    }
    {
        for (i = n; i >= 1; i--)
        for (j = 1; j <= i; j++)

```

```
{
printf("%d ", i);
}
printf("\n");
}
}
/* Pattern 4 */
void reverseTriangle(void)
{
int n, i, j;
printf("Enter number of rows: ");
scanf("%d", &n);
if (n <= 0)
{
printf("Error: Number of rows must be positive.\n");
return;
}
for (i = n; i >= 1; i--)
{
for (j = i; j >= 1; j--)
{
printf("%d ", j);
}
printf("\n");
}
}
```

4. Execution

For choice 1

```
===== DECREASING PATTERNS =====
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
Enter your choice: 1
Enter number of rows: 4
* * * *
* * *
* *
*
```

For choice 2

```
===== DECREASING PATTERNS =====
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
Enter your choice: 2
Enter number of rows: 4
1 2 3 4
1 2 3
1 2
1
```

For choice 3

```
===== DECREASING PATTERNS =====
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
Enter your choice: 3
Enter number of rows: 4
4 4 4 4
3 3 3
2 2
1
```

```
===== DECREASING PATTERNS =====
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
Enter your choice: 4
Enter number of rows: 4
4 3 2 1
3 2 1
2 1
1
```